SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO4: *Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization.(BL3,BL4,BL5)*

Assessment Tool Weightage: 40%

Dr G. A. SENTHIL

QUESTIONS: 25 **Total Marks: 50**

Annexure A

Experiments

DURATION: 4 HRS (2 Sessions)

Model-Based RL Approach

1. Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.

   (4 Marks)

2. Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.

   (4 Marks)

3. Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.

   (4 Marks)

4. Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.

   (4 Marks)

5. Prepare a concept map comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning.

(4 Marks)

Analytic Gradient Computation

6. Draw a flowchart explaining the steps involved in analytic gradient computation for policy optimization.

(4 Marks)

7. Design a concept map showing how gradients are computed and propagated during policy learning.

(4Marks)

8. Draw a block diagram illustrating gradient computation, parameter updates, and reward optimization in Reinforcement Learning.

(4 Marks)

9. Prepare a flowchart explaining gradient-based optimization using backpropagation in Reinforcement Learning.

(4 Marks)

10. Draw a concept map showing the relationship among policy parameters, gradients, optimization algorithms, and rewards.

(4 Marks)

Sampling-Based Planning

11. Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

(4 Marks)

12. Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

(4 Marks)

13. Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.

(4 Marks)

14. Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

(4 Marks)

15. Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

(4 Marks)

Model-Based Data Generation

16. Draw a flowchart illustrating synthetic data generation using a learned environment model.

(4 Marks)

17. Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.

(4 Marks)

18. Draw a block diagram explaining model-generated experience replay in Reinforcement Learning.

(4 Marks)

19. Design a flowchart showing how simulated experiences improve sample efficiency during training.

(4 Marks)

20. Draw a concept map illustrating the advantages and limitations of model-generated training data.

(4 Marks)

Value-Equivalence Prediction and Policy Optimization

21. Draw a flowchart explaining value-equivalence prediction for estimating future rewards.

(4 Marks)

22. Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.

(4 Marks)

23. Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

(4 Marks)

24. Design a flowchart explaining iterative policy optimization using learned environment dynamics.

(4 Marks)

25. Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

(4 Marks)

Code of Conduct:

I G.PRANATHI [192472237]certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

EXPERIMENTS

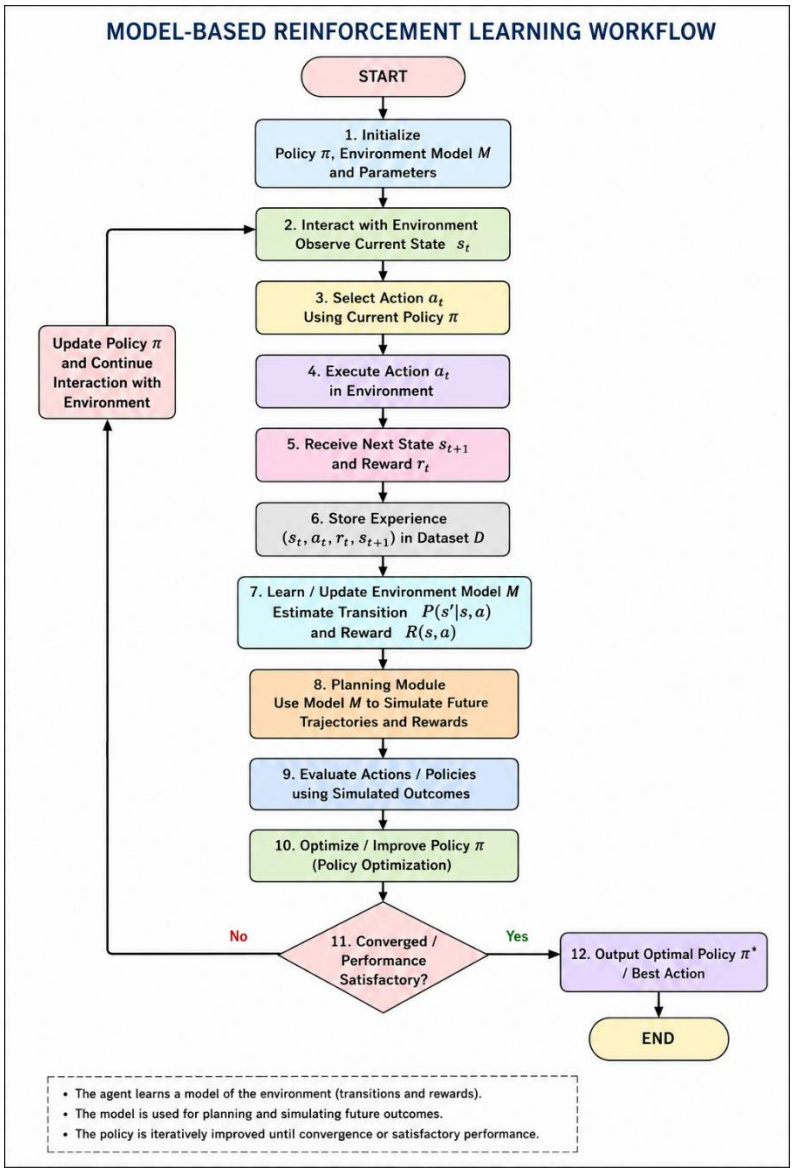
Model-Based RL Approach

Q1. Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.

Answer:

A **Model-Based Reinforcement Learning (MBRL)** agent learns or uses a model of the environment to predict future states and rewards. The learned model is then used for planning and policy optimization.

Flowchart :



Explanation

The workflow consists of the following steps:

1. **Initialization:** The agent initializes its policy and environment model.
2. **State Observation:** The agent observes the current state .
3. **Action Selection:** An action is selected using the current policy .
4. **Environment Interaction:** The action is performed in the environment.
5. **Receive Feedback:** The environment returns a reward and next state .
6. **Model Learning:** The agent updates the transition model and reward model .
7. **Planning:** The learned model is used to predict possible future states and rewards.
8. **Action Evaluation:** Actions are evaluated using value functions or the Bellman equation.
9. **Policy Optimization:** The policy is updated toward actions having higher expected returns.
10. **Iteration:** The process repeats until the policy converges or the desired goal is reached.

Key Equation

The Bellman optimality equation used during planning is:

where:

- = optimal value of state
- = reward for taking action
- = discount factor
- = transition probability

Conclusion

Thus, a Model-Based RL agent follows the cycle:

Observe → Act → Receive Reward → Learn Model → Plan → Evaluate → Optimize Policy → Repeat

This enables the agent to **predict future outcomes and improve its policy using the learned model of the environment**.

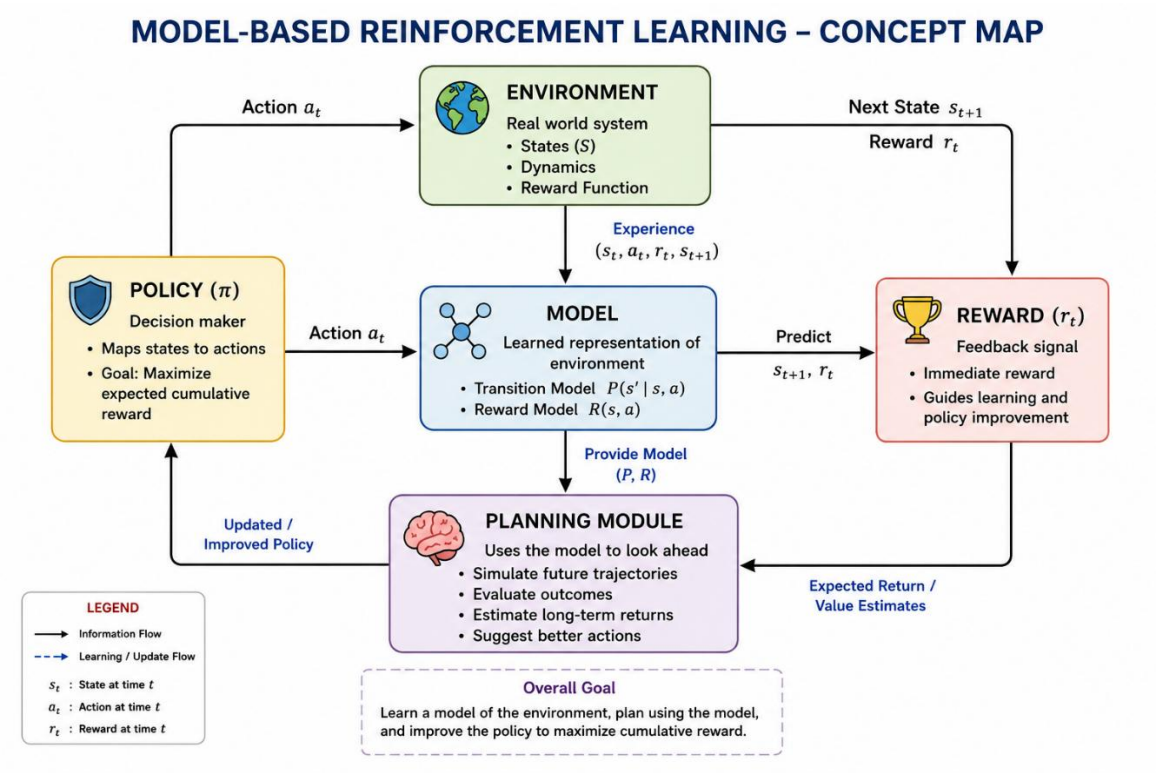
---

2. Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.

Answer:

Model-Based Reinforcement Learning consists of five major components: **Environment, Model, Planning Module, Policy, and Reward**. Their relationship can be represented as follows:

Concept :



Relationship between components

- **Environment:** Provides the current state and responds to the agent's actions.
- **Reward:** Provides feedback indicating how good or bad an action was.
- **Model:** Learns the transition dynamics and reward function .
- **Planning Module:** Uses the learned model to predict future states and rewards and evaluate possible actions.
- **Policy:** Selects the best action based on the planning results.
- **Policy Improvement:** The planning module improves the policy, creating a continuous **planning → policy → environment** cycle.

Key relationship

The **Reward** provides feedback to the agent and is incorporated into the learned model for better planning and policy optimization.

Conclusion

Thus, Model-Based RL combines **environment interaction, model learning, planning, reward feedback, and policy improvement** to select actions that maximize the expected cumulative reward.

3.Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.

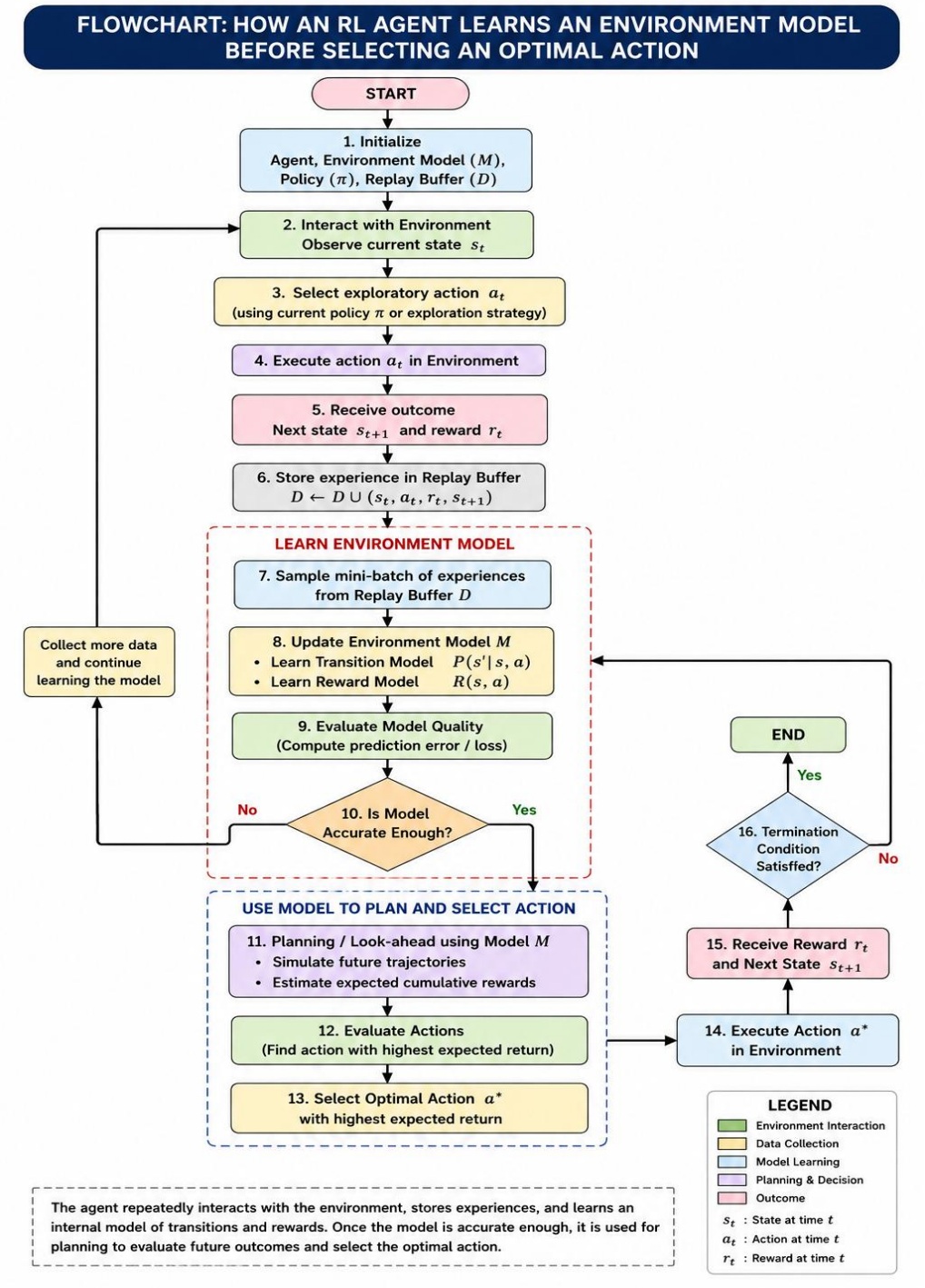
Answer :

In **Model-Based Reinforcement Learning**, the agent first interacts with the environment and collects experiences in the form of **state, action, reward, and next state**. It uses these experiences to learn the **transition model** and **reward model**, and then uses the learned model to predict future outcomes and select the optimal action.

Explanation

1. **Initialize RL Agent and Environment Model:** The agent and its model are initialized.
2. **Observe State:** The agent observes the current state .
3. **Select Action:** The agent selects an action .
4. **Interact with Environment:** The selected action is performed in the environment.
5. **Receive Feedback:** The environment provides reward and next state .
6. **Store Experience:** The transition is stored.
7. **Update Model:** The agent learns the transition model and reward model .
8. **Check Model:** If the model is not sufficiently learned, more experience is collected and the model is updated.
9. **Prediction and Planning:** Once learned, the model predicts future states and rewards.
10. **Evaluate Actions:** Possible actions are evaluated using the learned model.
11. **Select Optimal Action:** The action with the **highest expected return** is selected and executed.

Flow Chart:



Conclusion

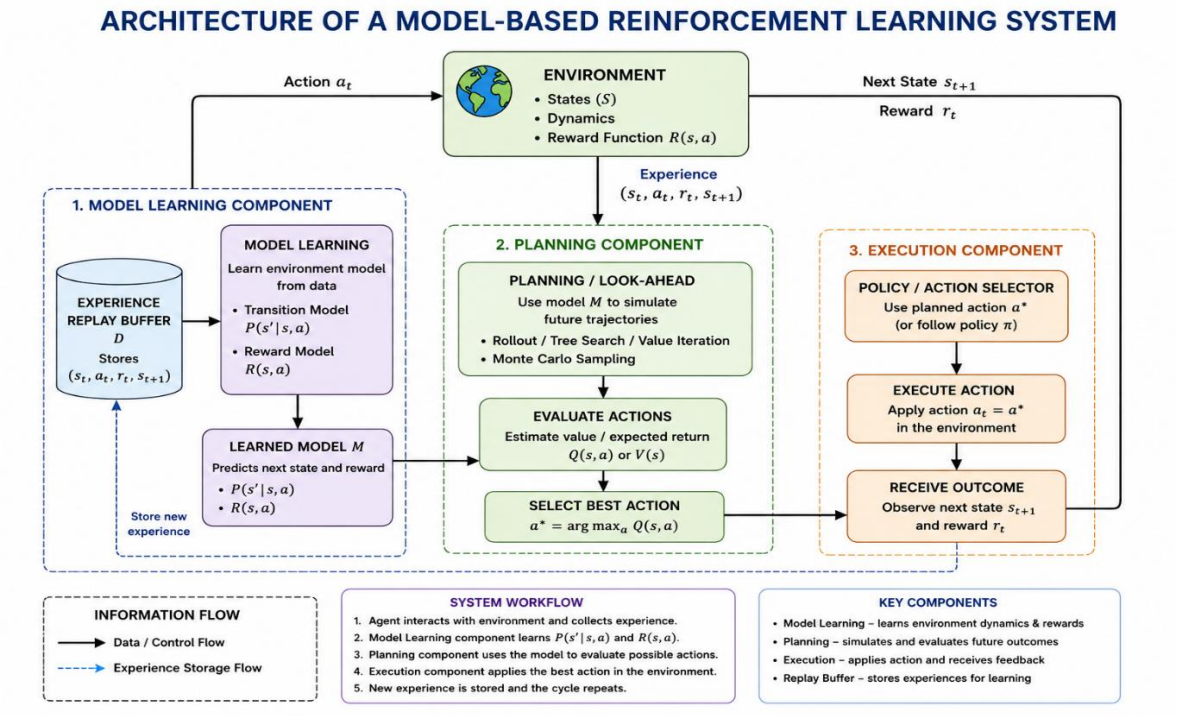
Thus, the RL agent **learns the environment model through experience first** and then uses **prediction and planning** to select an optimal action that maximizes the expected cumulative reward.

4. Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.

Answer

A **Model-Based Reinforcement Learning (MBRL)** system consists of three major stages: **Model Learning, Planning, and Execution**. The agent learns the environment model from experience, uses the learned model to plan future actions, and then executes the selected action in the real environment.

Architecture



Explanation

1. **Environment:** The agent interacts with the real environment and receives states and rewards.
2. **Model Learning:** The agent uses collected experience to learn the **transition model** and **reward model** .
3. **Planning:** The learned model is used to simulate future outcomes and evaluate possible actions.
4. **Action Selection:** Planning identifies the action with the highest expected return.
5. **Execution:** The selected optimal action is executed in the environment.
6. **Feedback Loop:** The new experience is again used to improve the environment model.

Key Architecture

Environment → Model Learning → Planning → Execution → Environment

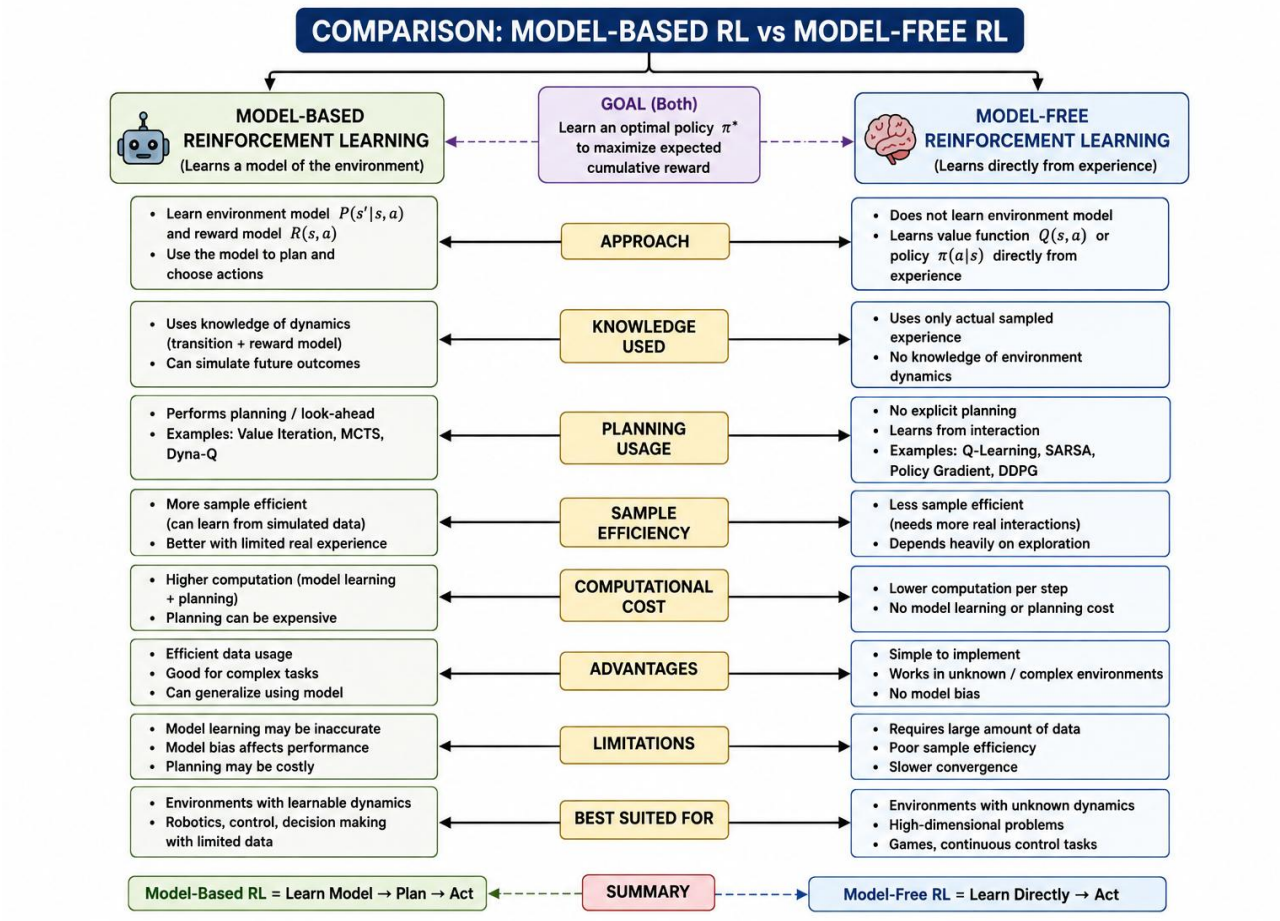
This continuous feedback loop allows the agent to **learn, plan, execute, and improve its decisions**.

---

5. Prepare a Concept Map Comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning

Answer

Model-Based Reinforcement Learning (MBRL) learns an internal model of the environment, including state transitions and rewards, and uses it for planning. **Model-Free Reinforcement Learning (MFRL)** learns the policy or value function directly from interactions without explicitly building an environment model.



Explanation

1. Model-Based RL

- Learns the **transition model** and **reward model** .
- Uses the learned model to **predict future states and rewards**.
- Performs **planning** before selecting actions.

- Usually provides better **sample efficiency** because experience can be reused through simulation.
- Model errors can lead to incorrect planning and decisions.

2. Model-Free RL

- Does not explicitly learn a model of the environment.
- Directly learns a **value function, Q-function, or policy** from experience.
- Does not perform model-based look-ahead planning.
- Simpler to implement and can work well in complex environments.
- Usually requires more real-world experience or samples.

Key Comparison

Feature	**Model-Based RL**	**Model-Free RL**
Environment model	Required/learned	Not required
Planning	Yes	No explicit planning
Learning	Model + policy/value	Policy/value directly
Sample efficiency	Generally higher	Generally lower
Complexity	Higher	Lower
Examples	Dyna-Q, Value Iteration	Q-Learning, SARSA

Conclusion

Model-Based RL = Learn the environment → Plan → Act

Model-Free RL = Experience → Learn directly → Act

Both approaches aim to learn a policy that **maximizes expected cumulative reward**, but they differ mainly in whether an explicit environment model is learned and used for planning.

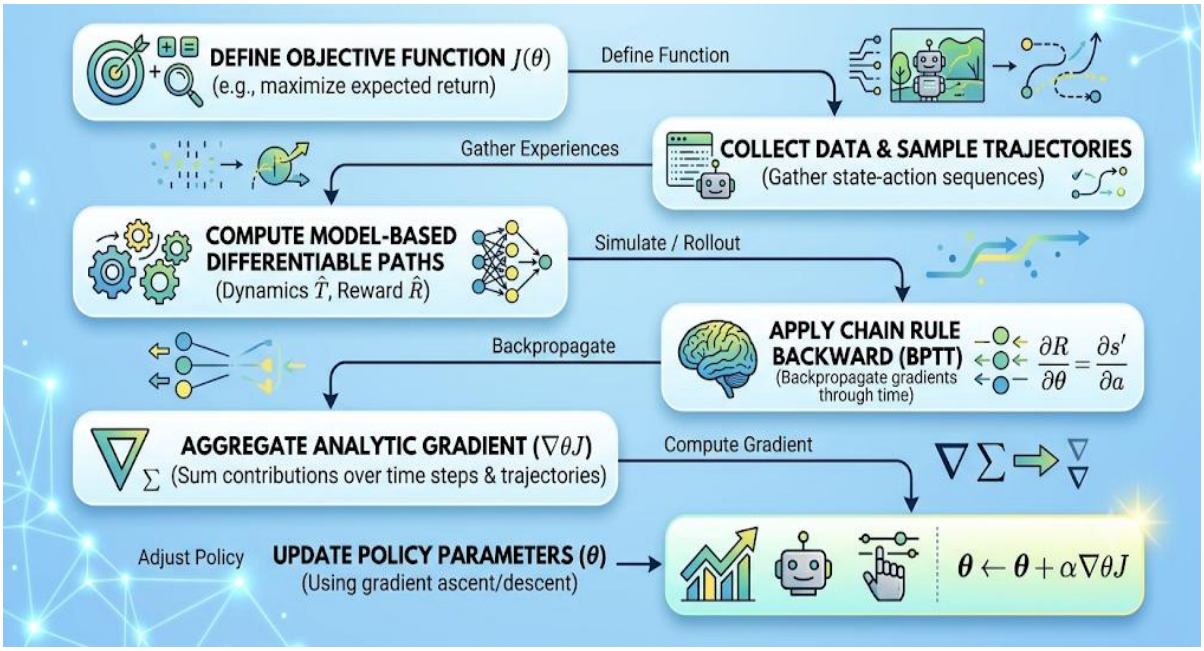
Analytic Gradient Computation

6. Draw a Flowchart Explaining the Steps Involved in Analytic Gradient Computation for Policy Optimization

Small Explanation

Analytic gradient computation calculates the gradient of the expected return with respect to the policy parameters. The gradient is then used to update the policy so that the agent gradually improves its performance.

Flowchart



Explanation

1. **Initialize Policy:** Initialize the policy parameters .
2. **Collect Trajectories:** Run the current policy in the environment and collect states, actions, and rewards.
3. **Compute Returns:** Calculate the cumulative discounted return for each time step.
4. **Compute Policy Gradient:** Use the policy gradient theorem to analytically calculate the gradient.
5. **Estimate Gradient:** Average the gradient estimates obtained from collected trajectories.
6. **Update Policy Parameters:** Update using the gradient and learning rate :
7. **Check Convergence:** If the policy has converged, stop. Otherwise, collect new trajectories and repeat the process.

Key Sequence

Initialize Policy → Collect Trajectories → Compute Returns → Compute Gradient → Estimate Gradient → Update Policy → Check Convergence → Repeat/End

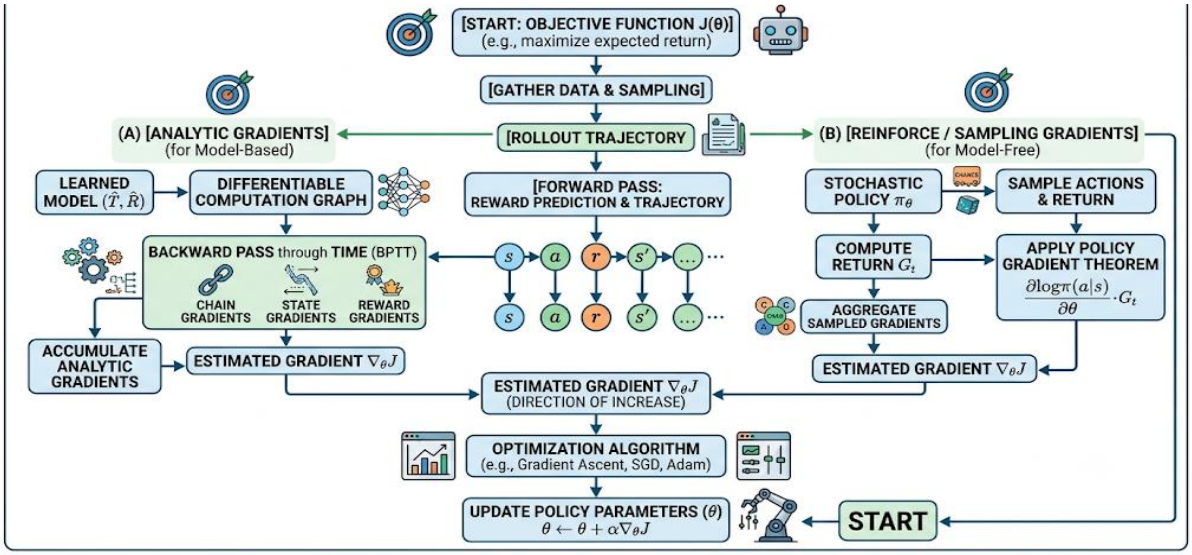
---

7. Design a Concept Map Showing How Gradients Are Computed and Propagated During Policy Learning

Small Explanation

In **policy learning**, the agent uses collected experiences to calculate how changing the policy parameters affects the expected reward. The gradient is then **computed, propagated through the policy, and used to update the parameters** so that future actions produce higher expected returns.

Concept Map :



Explanation

1. **Environment Interaction:** The agent follows the current policy and collects states, actions, and rewards.
2. **Trajectory Collection:** Experiences are stored as trajectories or episodes.
3. **Return Calculation:** The cumulative discounted reward is calculated.
4. **Gradient Computation:** The policy gradient is calculated using the relationship between action probabilities and returns.
5. **Gradient Propagation:** The gradient information is propagated back to the policy parameters .
6. **Parameter Update:** The parameters are updated using gradient ascent:
7. **Policy Improvement:** The updated parameters produce a better policy that aims to increase expected cumulative reward.
8. **Iteration:** The process repeats until the policy reaches satisfactory performance or converges.

Key Flow

Environment → Trajectories → Returns → Gradient Computation → Gradient Propagation → Parameter Update → Improved Policy → Environment

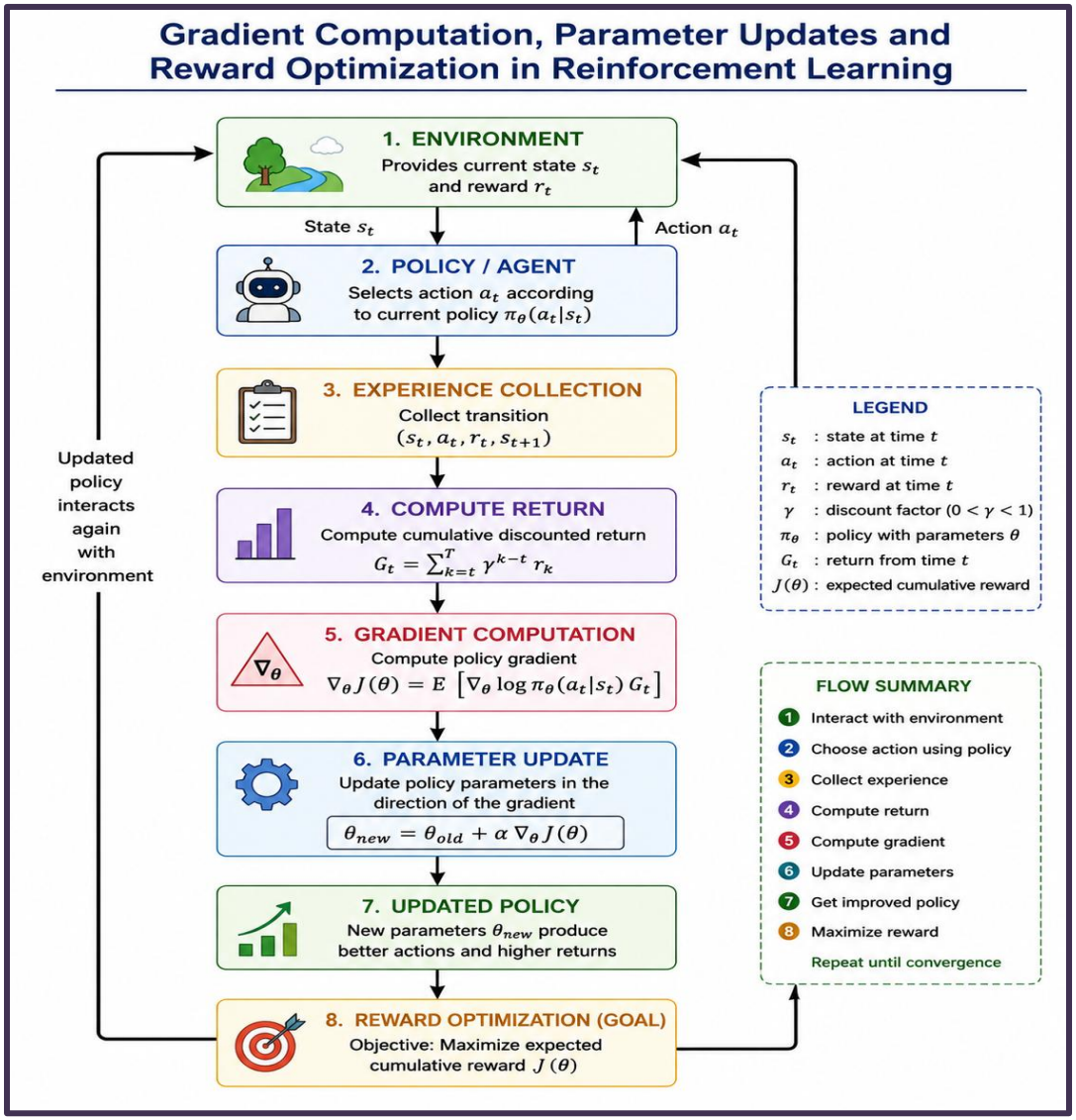
---

8. Draw a Block Diagram Illustrating Gradient Computation, Parameter Updates, and Reward Optimization in Reinforcement Learning

Small Explanation

In policy-gradient Reinforcement Learning, the agent uses **rewards from the environment** to compute a gradient that indicates how the policy parameters should change. The parameters are then updated to **maximize the expected cumulative reward**.

Block Diagram



Explanation

1. **Environment:** Provides the current state and reward after the agent takes an action.

2. **Policy/Agent:** Selects an action according to the current policy.
3. **Experience Collection:** Stores the state, action, reward, and next state.
4. **Return Computation:** Calculates the cumulative or discounted reward obtained from the experience.
5. **Gradient Computation:** Calculates , which shows the direction in which policy parameters should change.
6. **Parameter Update:** Updates the parameters using:
7. **Updated Policy:** The new parameters produce an improved policy.
8. **Reward Optimization:** The objective is to maximize the expected cumulative reward.
9. **Repeat:** The process continues with new interactions until the policy converges.

Key Flow

Environment → Experience → Return → Gradient Computation → Parameter Update → Improved Policy → Higher Reward → Repeat

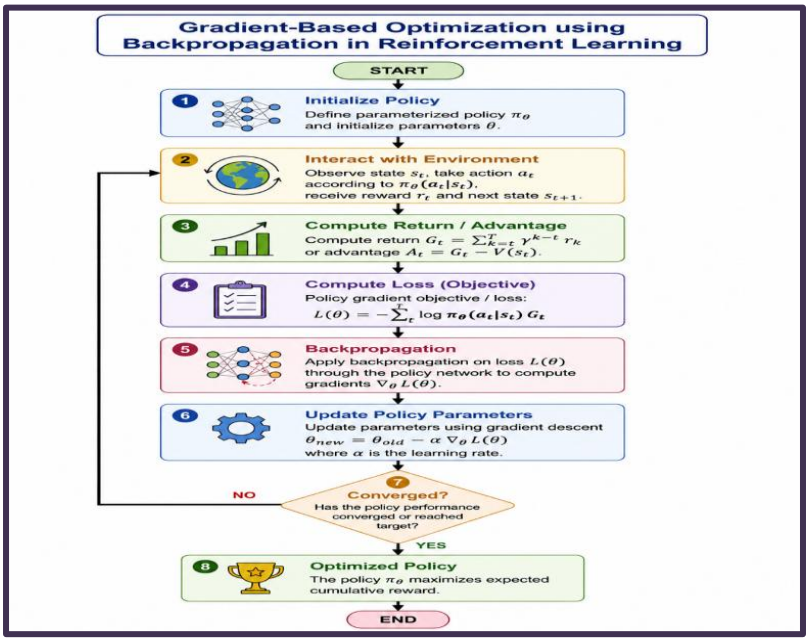
---

9. Prepare a Flowchart Explaining Gradient-Based Optimization Using Backpropagation in Reinforcement Learning

Small Explanation

In Reinforcement Learning, **backpropagation** is used to calculate how the policy parameters affect the objective or loss. The calculated gradients are then used to update the parameters and improve the policy.

Flow Chart :



Explanation

1. **Initialize Policy:** Define the parameterized policy and initialize its parameters .
2. **Interact with Environment:** The agent observes states, selects actions, and receives rewards.
3. **Compute Returns/Advantages:** Calculate the cumulative discounted return or advantage .
4. **Compute Loss:** Form the policy objective/loss using the collected experience.
5. **Backpropagation:** Propagate the error/loss backward through the policy network to calculate gradients .
6. **Parameter Update:** Update the policy parameters using gradient descent:
7. **Repeat:** Collect new experiences using the updated policy and repeat the process.
8. **Optimized Policy:** Continue until the policy converges and produces actions that maximize expected cumulative reward.

Key Flow

Initialize Policy → Environment Interaction → Compute Returns → Compute Loss → Backpropagation → Gradient Computation → Parameter Update → Repeat → Optimized Policy

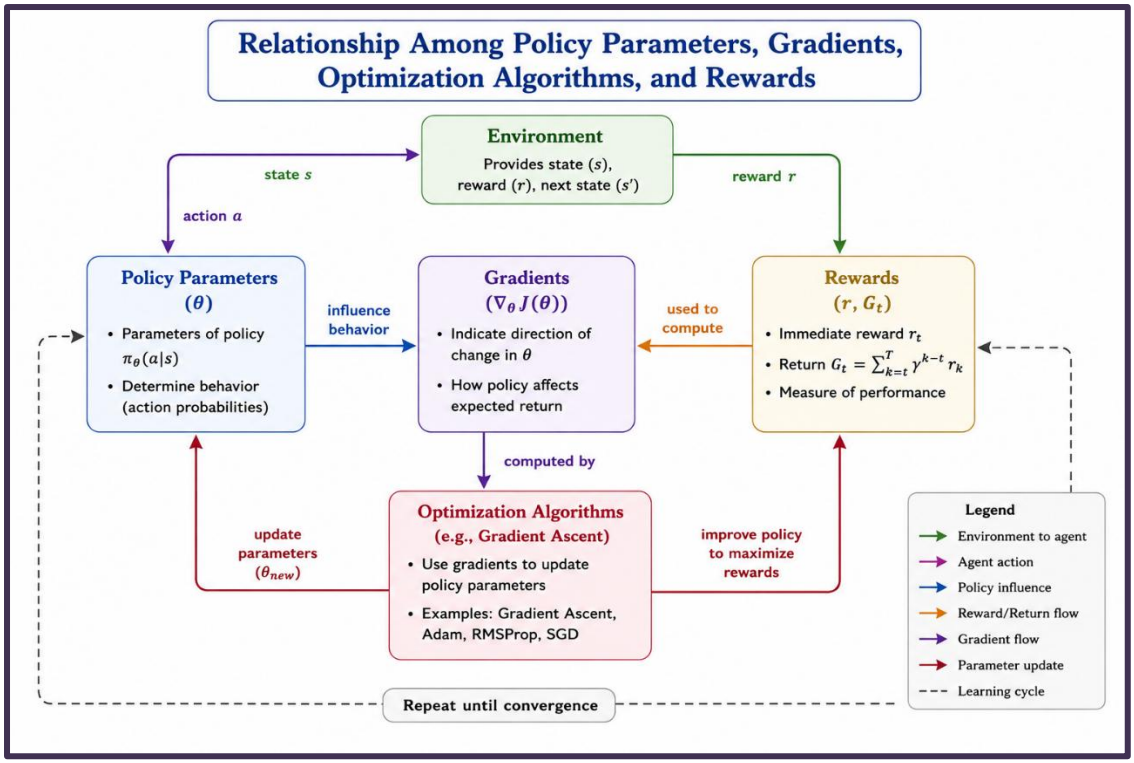
---

10. Concept Map Showing the Relationship Among Policy Parameters, Gradients, Optimization Algorithms, and Rewards

Small Explanation

In policy-based Reinforcement Learning, **policy parameters** control the agent's behavior. The **rewards** received from the environment are used to calculate **gradients**, which indicate how the parameters should change. **Optimization algorithms** use these gradients to update the parameters and improve the expected reward.

Concept Map :



Explanation

- **Policy Parameters ()** determine the actions selected by the policy.
- **Rewards ()** measure how well the agent performs.
- **Gradients** indicate the direction in which policy parameters should change to increase expected reward.
- **Optimization Algorithms** such as Gradient Ascent, SGD, and Adam use the gradients to update the policy parameters.
- The updated parameters produce an **improved policy**, which aims to obtain higher rewards.

Key Relationship

Policy Parameters → Actions → Rewards → Gradients → Optimization Algorithm → Updated Policy Parameters → Higher Rewards

This process is repeated until the policy reaches satisfactory performance or converges.\

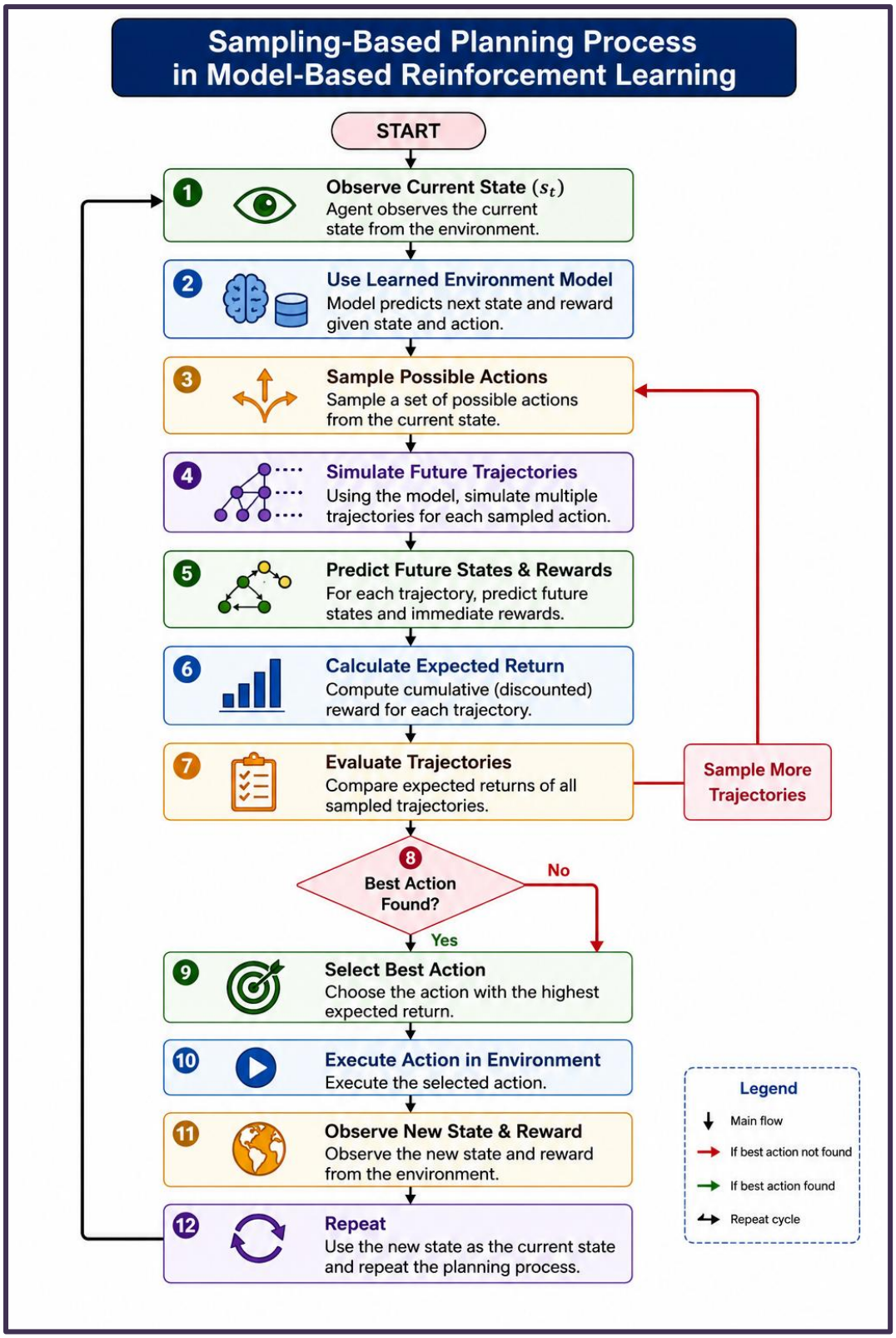
Sampling-Based Planning

11. Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

Small Explanation

Sampling-based planning uses a learned environment model to sample possible actions and simulate future trajectories. The agent compares their expected rewards and selects the action with the highest expected return.

Flowchart



Explanation

1. **Observe State:** The agent observes the current state.
2. **Use Model:** The learned model predicts future outcomes.
3. **Sample Actions:** Several possible actions are selected.
4. **Simulate:** Future trajectories are generated using the model.
5. **Predict:** Future states and rewards are estimated.
6. **Calculate Return:** Expected cumulative rewards are calculated.
7. **Evaluate:** The sampled trajectories are compared.
8. **Select:** The action with the highest expected return is chosen.
9. **Execute:** The selected action is performed in the environment.
10. **Repeat:** The new state and reward are used for the next planning cycle.

Key Flow:

State → Sample → Simulate → Predict → Evaluate → Select → Execute → Repeat

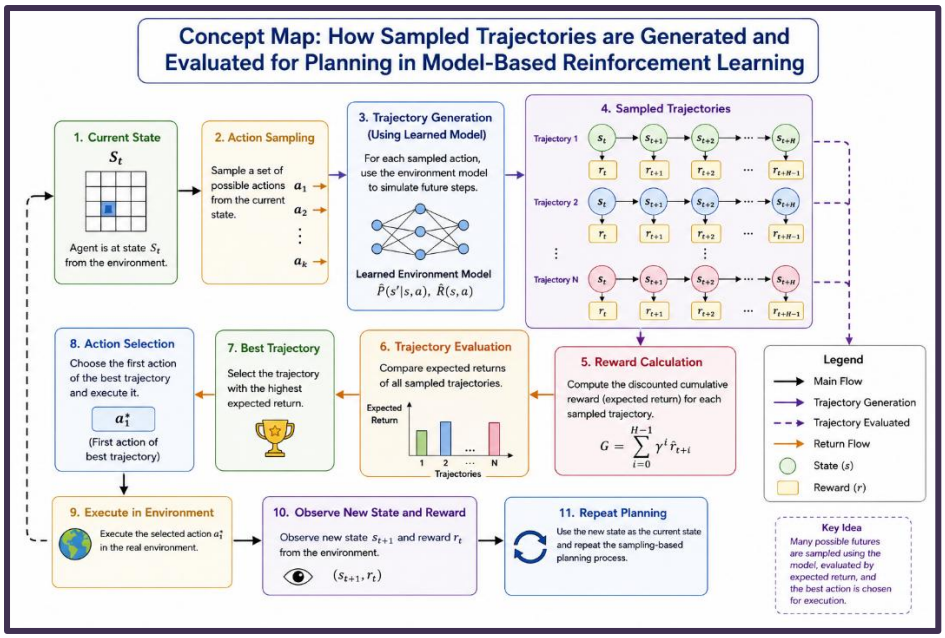
---

12. Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

Small Explanation

In **sampling-based planning**, the agent uses the learned environment model to generate multiple possible future trajectories. Each trajectory is evaluated using its predicted rewards, and the best trajectory is used to select an action.

Concept Map



Explanation

- **Current State:** Planning starts from the agent's current state .
- **Action Sampling:** Different possible actions are sampled.
- **Trajectory Generation:** The learned model simulates future states and rewards for each action.
- **Sampled Trajectories:** Multiple possible paths are generated.
- **Reward Calculation:** The expected cumulative reward is calculated for each trajectory.
- **Trajectory Evaluation:** The trajectories are compared based on their expected returns.
- **Best Trajectory:** The trajectory with the highest expected return is selected.
- **Action Selection:** The first action of the best trajectory is executed in the environment.

Key Relationship

Current State → Sample Actions → Generate Trajectories → Predict Rewards → Evaluate Trajectories → Select Best Trajectory → Execute Action

This process is repeated with the newly observed state for continuous planning.

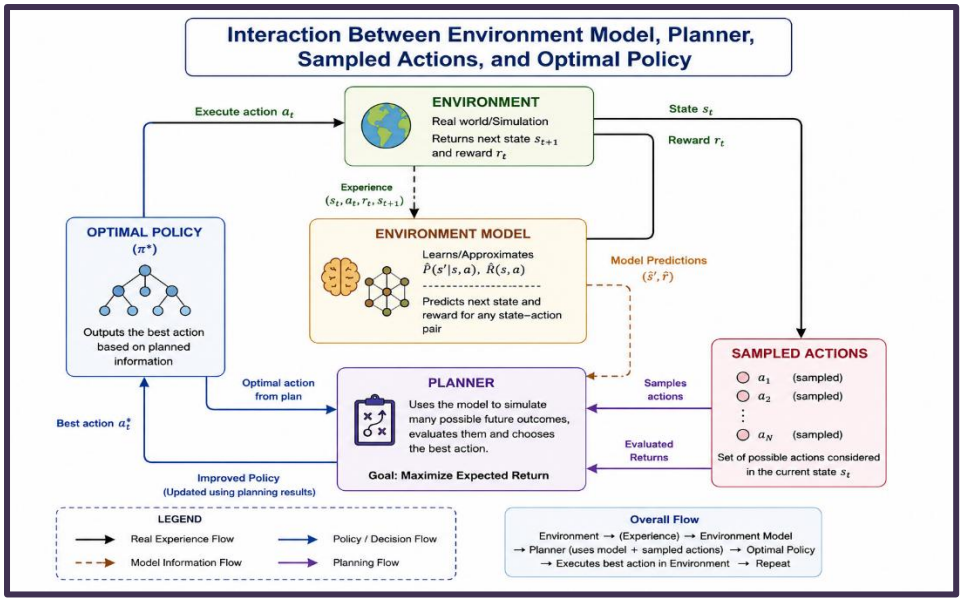
---

13. Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.

Small Explanation

In Model-Based Reinforcement Learning, the **environment model** predicts future outcomes, the **planner** evaluates different sampled actions, and the **optimal policy** selects the best action to execute in the environment.

Block Diagram



Explanation

1. **Environment:** Provides the current state and reward based on the executed action.
2. **Environment Model:** Learns from experience and predicts future states and rewards.
3. **Sampled Actions:** Different possible actions are generated for the current state.
4. **Planner:** Uses the environment model to simulate and evaluate the sampled actions.
5. **Optimal Policy:** Selects the action with the highest expected return.
6. **Execution:** The selected action is executed in the environment.
7. **Feedback:** The new state and reward are used to continue the learning and planning cycle.

Key Flow

Environment → Environment Model → Planner → Sampled Actions → Optimal Policy → Environment

This cycle continues until the agent learns an effective policy.

---

14. Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

Small Explanation

Monte Carlo sampling uses repeated random simulations or **rollouts** to estimate the expected return of possible actions. The action with the highest average return is selected.

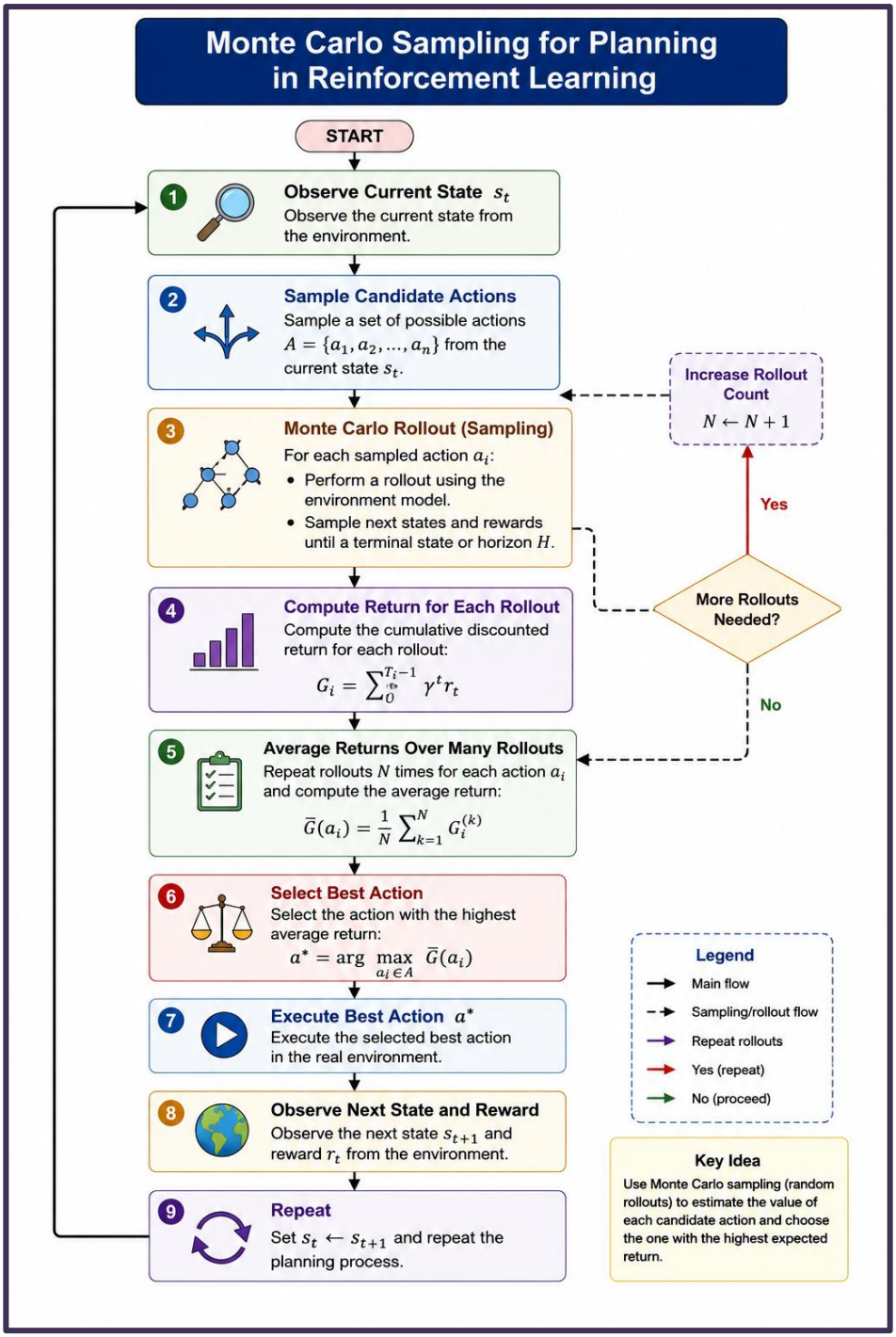
Explanation

1. **Observe State:** Observe the current state .
2. **Sample Actions:** Generate possible candidate actions.
3. **Monte Carlo Rollout:** Simulate each action through multiple random trajectories.
4. **Calculate Return:** Calculate the cumulative reward for each rollout.
5. **Average Returns:** Average the returns obtained from multiple rollouts.
6. **Select Best Action:** Choose the action with the highest average return.
7. **Execute Action:** Perform the selected action in the environment.
8. **Repeat:** Observe the new state and repeat the planning process.

Key Flow:

State → Sample Actions → Rollouts → Calculate Returns → Average Returns → Best Action → Execute → Repeat

Flow Chart :



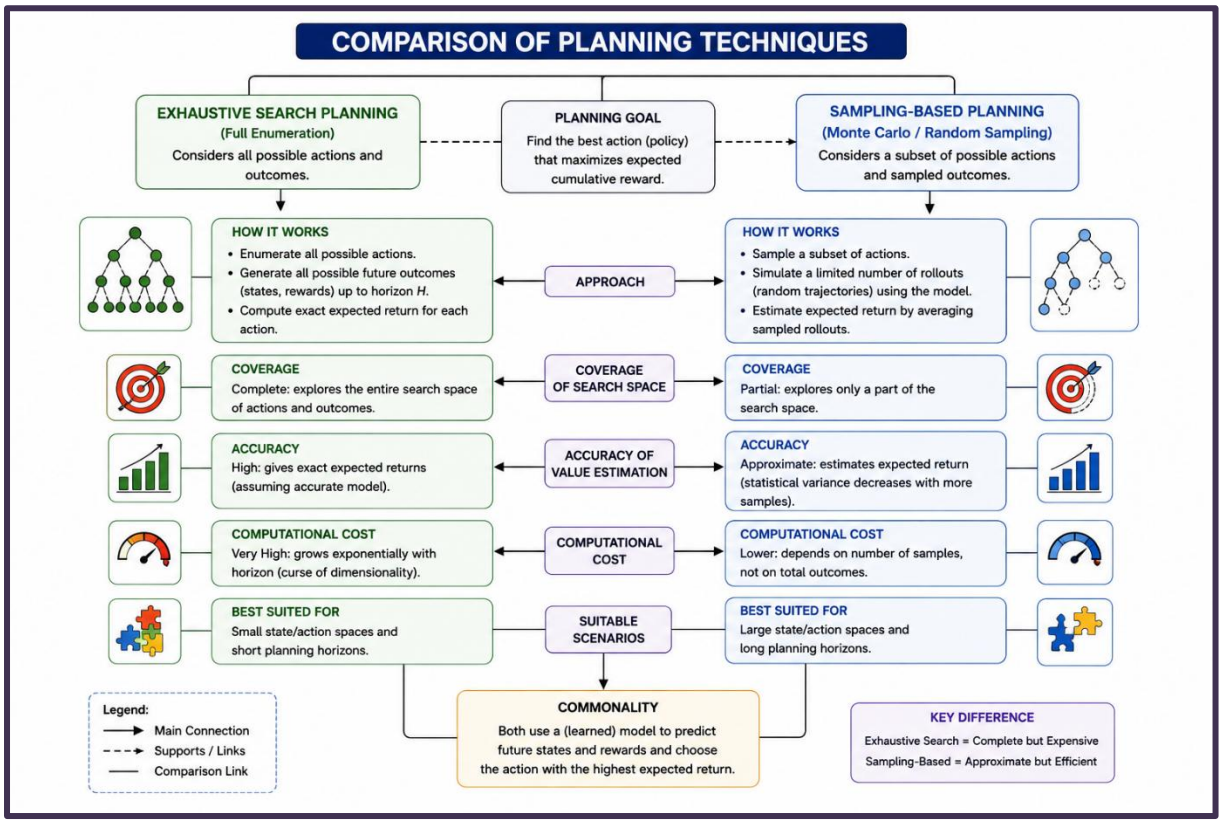
---

15. Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

Small Explanation

Exhaustive Search Planning evaluates all possible actions and future outcomes, while **Sampling-Based Planning** evaluates only a selected number of sampled actions and trajectories. Both aim to find the action with the highest expected reward.

Concept Map



Explanation

Exhaustive Search Planning

- Considers **all possible actions and outcomes**.
- Provides more complete and accurate planning.
- Has **high computational cost**.
- Suitable for small state spaces and short planning horizons.

Sampling-Based Planning

- Considers a **subset of actions and trajectories**.
- Provides an approximate estimate of expected returns.
- Has **lower computational cost**.
- Suitable for large state spaces and long planning horizons.

Key Difference

Exhaustive Search	**Sampling-Based**
Complete search	Partial search
More accurate	Approximate
Computationally expensive	Computationally efficient
Small state spaces	Large state spaces

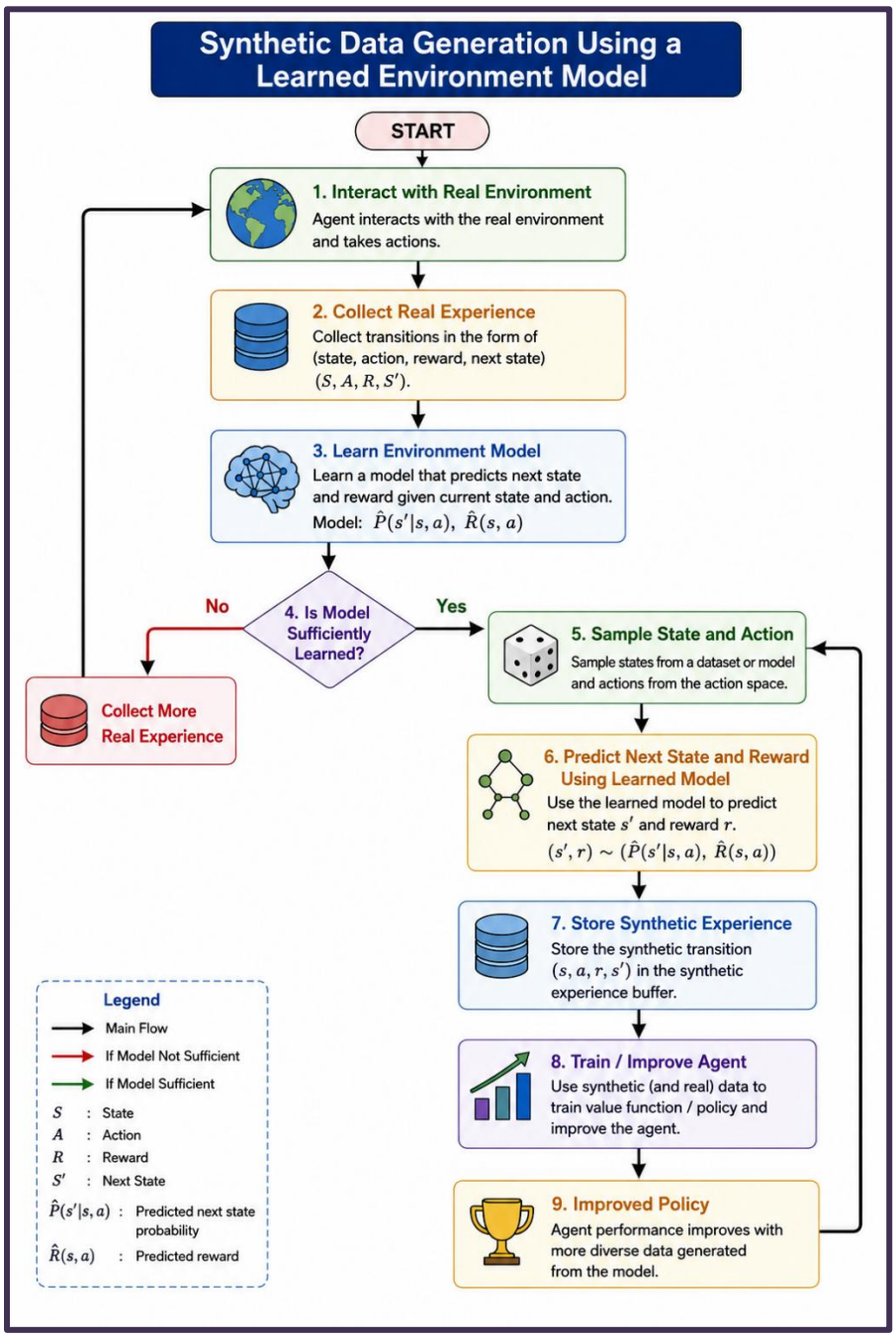
Model-Based Data Generation

16. Draw a flowchart illustrating synthetic data generation using a learned environment model.

Small Explanation

Synthetic data generation uses a learned environment model to simulate experiences without repeatedly interacting with the real environment. This generates additional **state, action, reward, and next-state** data for training the RL agent.

Flowchart



Explanation

1. **Collect Experience:** The agent interacts with the real environment and collects transitions.
2. **Learn Model:** The collected data is used to learn the environment model.
3. **Check Model:** If the model is not sufficiently learned, more real experience is collected.
4. **Generate Data:** The learned model generates synthetic experiences.
5. **Predict:** The model predicts the next state and reward for sampled state-action pairs.
6. **Store Data:** The generated experiences are stored as synthetic training data.
7. **Improve Agent:** The synthetic data is used to train and improve the RL agent.

Key Flow:

Real Experience → Learn Model → Generate Synthetic Data → Train Agent → Improved Policy

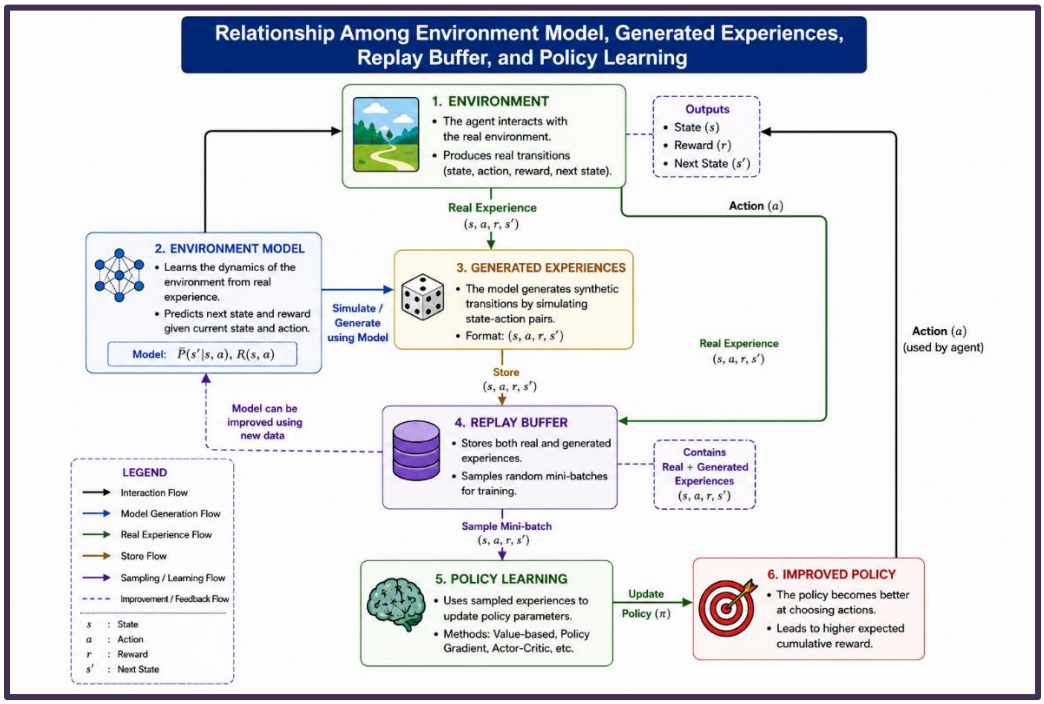
---

17. Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.

Small Explanation

In Model-Based RL, the **environment model** generates synthetic experiences from simulated states and actions. These experiences are stored in a **replay buffer** and used along with real experiences to train and improve the **policy**.

Concept Map



Explanation

- **Environment:** Provides real states, actions, and rewards.
- **Environment Model:** Learns the environment dynamics and generates synthetic experiences.
- **Generated Experiences:** Include state, action, reward, and next state .
- **Replay Buffer:** Stores real and generated experiences for later training.
- **Policy Learning:** Samples experiences from the replay buffer and updates the policy.
- **Improved Policy:** Performs better actions and interacts with the environment again.

Key Relationship

Environment → Model → Generated Experiences → Replay Buffer → Policy Learning → Improved Policy → Environment

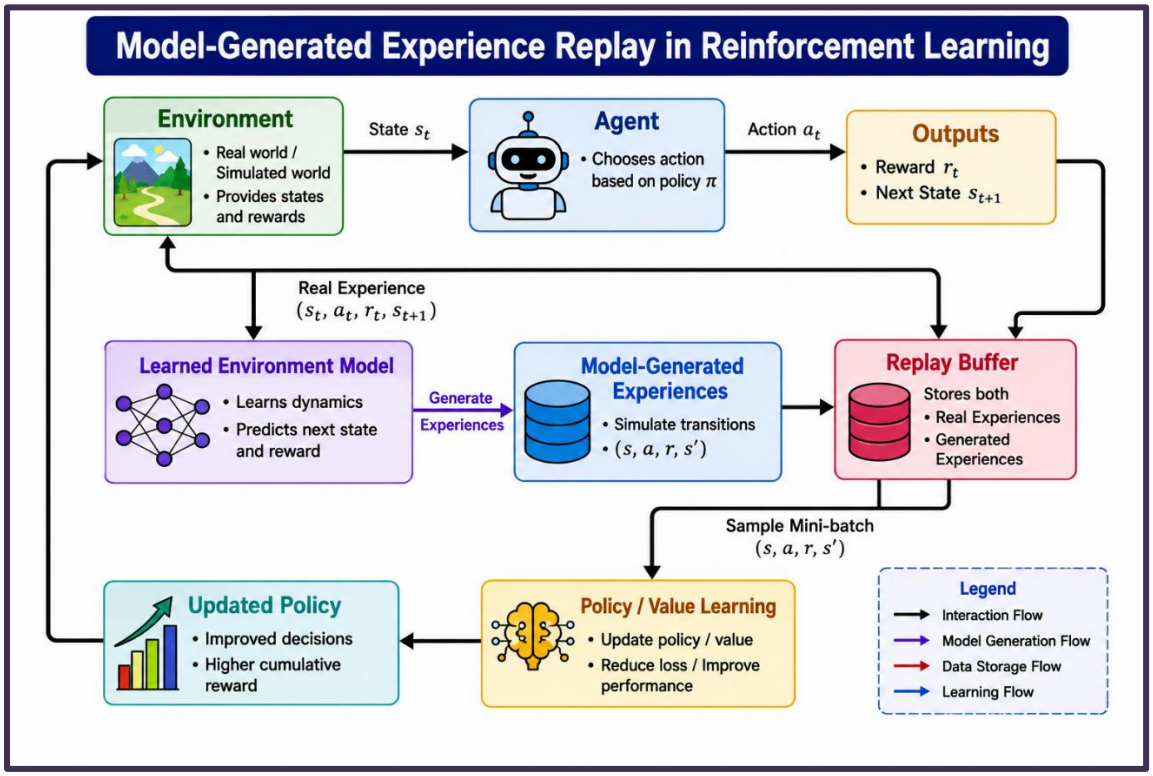
---

18. Draw a block diagram explaining model-generated experience replay in Reinforcement Learning.

Small Explanation

Model-generated experience replay uses a learned environment model to create additional synthetic experiences. These experiences are stored with real experiences in a **replay buffer** and used to improve the policy or value function.

Block Diagram



Explanation

1. **Environment:** Provides the current state and reward.
2. **Agent:** Selects an action using the current policy.
3. **Learned Environment Model:** Learns environment dynamics from real experiences.
4. **Generate Experiences:** The model simulates additional state-action-reward-next-state transitions.
5. **Replay Buffer:** Stores both **real and model-generated experiences**.
6. **Policy/Value Learning:** Samples mini-batches from the replay buffer to update the agent.
7. **Updated Policy:** The improved policy produces better actions and interacts with the environment again.

Key Flow

Environment → Learned Model → Generated Experiences → Replay Buffer → Policy Learning → Updated Policy → Environment

---

19. Design a flowchart showing how simulated experiences improve sample efficiency during training.

Small Explanation

Simulated experiences allow an RL agent to learn from additional data generated by an environment model. This reduces the need for frequent interaction with the real environment and improves **sample efficiency**.

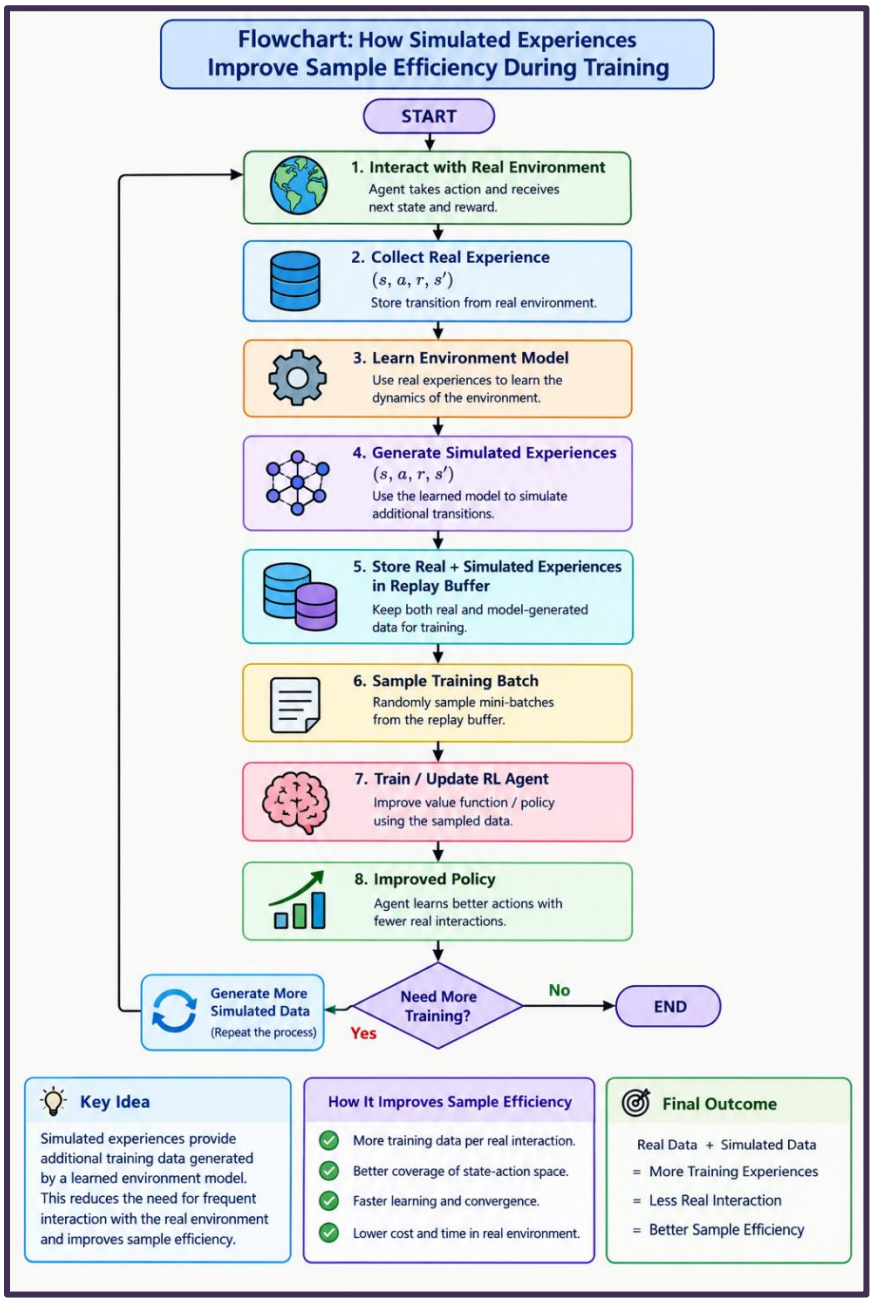
Explanation

1. **Real Interaction:** The agent collects experience from the real environment.
2. **Learn Model:** The collected data is used to learn an environment model.
3. **Simulation:** The model generates additional synthetic experiences.
4. **Replay Buffer:** Real and simulated experiences are stored together.
5. **Training:** The agent samples data and updates its policy or value function.
6. **Improved Policy:** The agent learns without requiring all experiences from the real environment.
7. **Repeat:** More simulated data can be generated when additional training is needed.

Key Idea

Real Data + Simulated Data → More Training Experiences → Less Real Interaction → Better Sample Efficiency

Flowchart



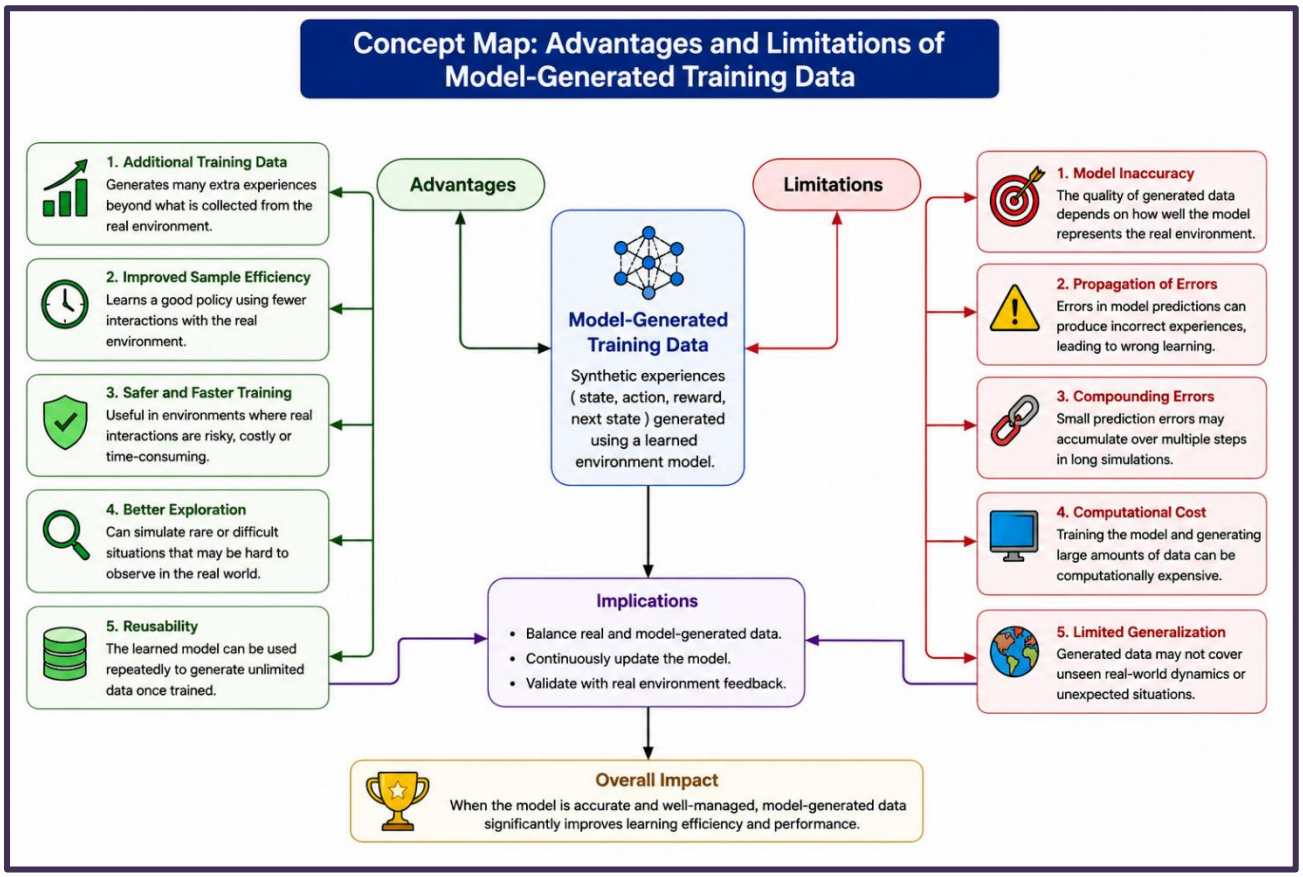
---

20. Draw a concept map illustrating the advantages and limitations of model-generated training data.

Small Explanation

Model-generated training data is synthetic experience produced by a learned environment model. It can provide extra training data and improve sample efficiency, but inaccurate model predictions may introduce errors.

Concept Map



Explanation

Advantages

- Provides **additional training data**.
- Reduces interaction with the **real environment**.
- Improves **sample efficiency**.
- Supports faster and safer training.
- Helps explore more possible state-action situations.

Limitations

- Depends on the **accuracy of the learned model**.
- Incorrect predictions can produce **unreliable data**.
- Model errors may accumulate during long simulations.
- Generating many simulations can require **computational resources**.
- Synthetic data may not fully represent the real environment.

Key Idea

Model-Generated Data → More Training Data → Better Sample Efficiency

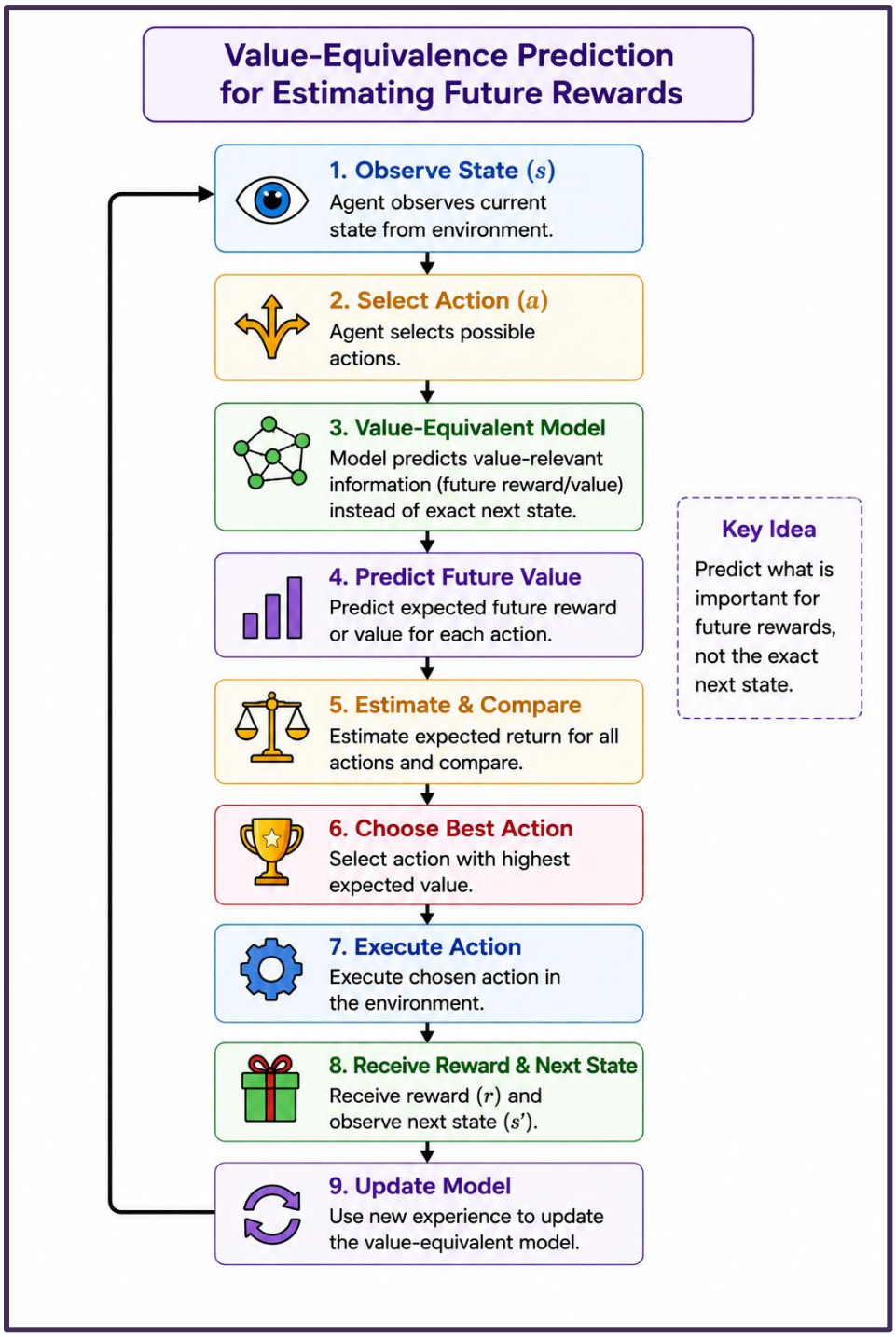
Value-Equivalence Prediction and Policy Optimization

21. Draw a flowchart explaining value-equivalence prediction for estimating future rewards.

Small Explanation

Value-equivalence prediction focuses on predicting the **future value or reward** needed for decision-making instead of accurately predicting every detail of the next environment state.

Flowchart



Explanation

1. **Observe State:** The agent observes the current state.
2. **Select Action:** Possible actions are considered.
3. **Use Model:** The value-equivalent model predicts future values or rewards.
4. **Estimate Return:** Expected future reward is calculated.
5. **Compare Actions:** Actions are evaluated using their predicted values.
6. **Select Best Action:** The action with the highest expected value is chosen.
7. **Execute:** The selected action is performed in the environment.
8. **Update:** New experience is used to improve the model.

Key Flow:

State → Action → Value Prediction → Expected Reward → Action Evaluation → Best Action → Update

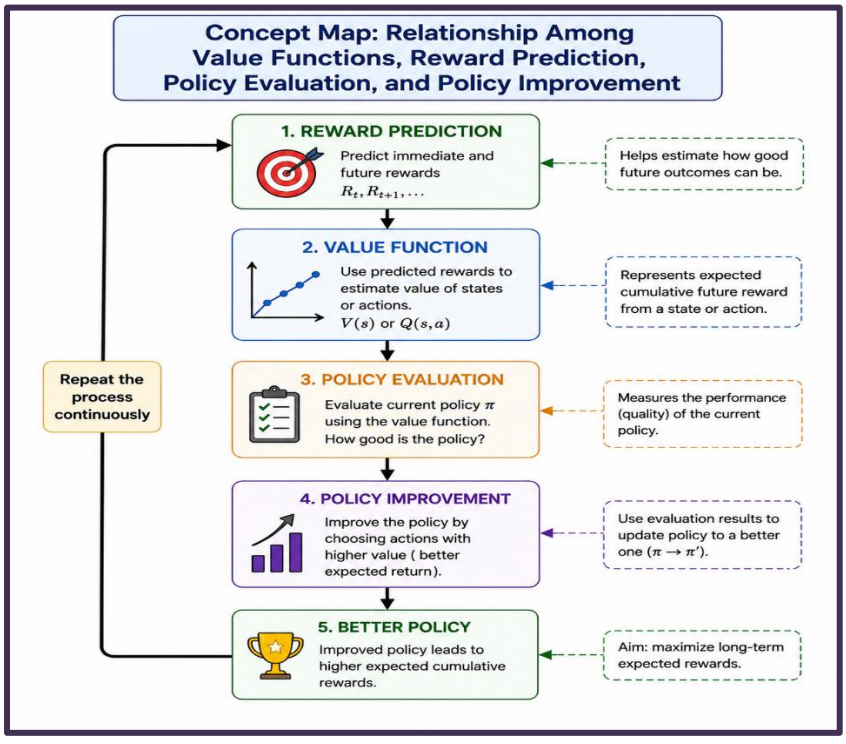
---

22. Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.

Small Explanation

In Reinforcement Learning, **reward prediction** helps estimate future rewards. These estimates are used by the **value function** to evaluate the current policy. The evaluation results guide **policy improvement** toward better actions.

Concept Map



Explanation

- **Reward Prediction:** Estimates immediate and future rewards.
- **Value Function:** Represents the expected cumulative reward from a state or action.
- **Policy Evaluation:** Uses value estimates to measure how well the current policy performs.
- **Policy Improvement:** Chooses better actions based on the value estimates.
- **Better Policy:** The improved policy aims to obtain higher cumulative rewards.

Key Relationship

Reward Prediction → Value Function → Policy Evaluation → Policy Improvement → Better Rewards → Repeat

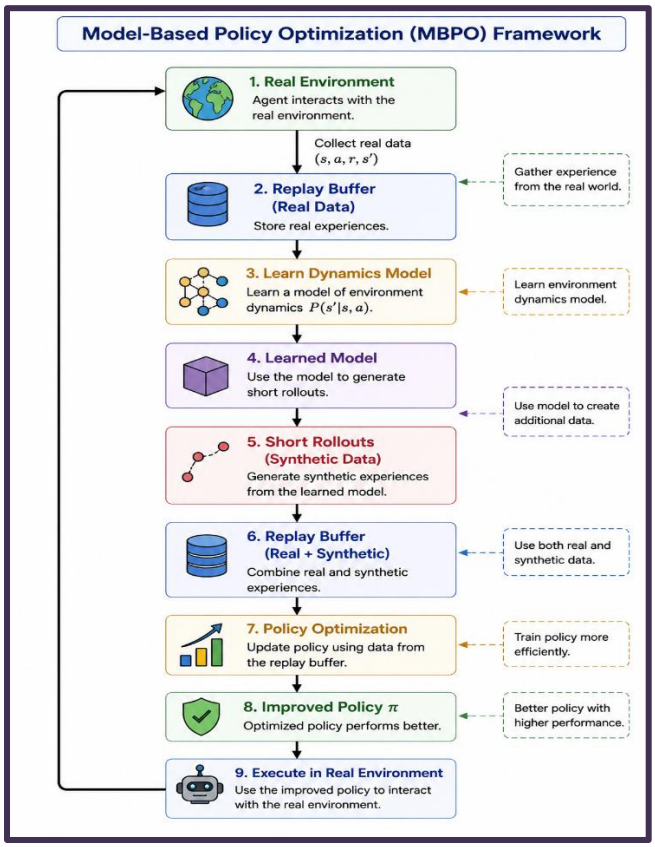
---

23. Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

Small Explanation

MBPO (Model-Based Policy Optimization) combines a learned environment model with model-free policy learning. The model generates short synthetic rollouts, which are added to real experience and used to improve the policy efficiently.

Block Diagram



Explanation

1. **Real Environment:** The agent interacts with the actual environment.
2. **Real Data:** State, action, reward, and next-state experiences are collected.
3. **Replay Buffer:** Real experiences are stored.
4. **Dynamics Model:** A model is learned from real experience.
5. **Short Rollouts:** The model generates short synthetic trajectories.
6. **Combined Data:** Real and synthetic experiences are stored in the replay buffer.
7. **Policy Optimization:** The policy is trained using the available experiences.
8. **Improved Policy:** The updated policy interacts with the real environment again.

Key Flow:

Real Data → Learn Model → Short Rollouts → Replay Buffer → Policy Optimization → Improved Policy → Real Environment

---

24. Design a flowchart explaining iterative policy optimization using learned environment dynamics.

Small Explanation

Iterative policy optimization repeatedly uses a learned environment model to predict outcomes, evaluate the current policy, and improve it until the policy becomes stable or performs well.

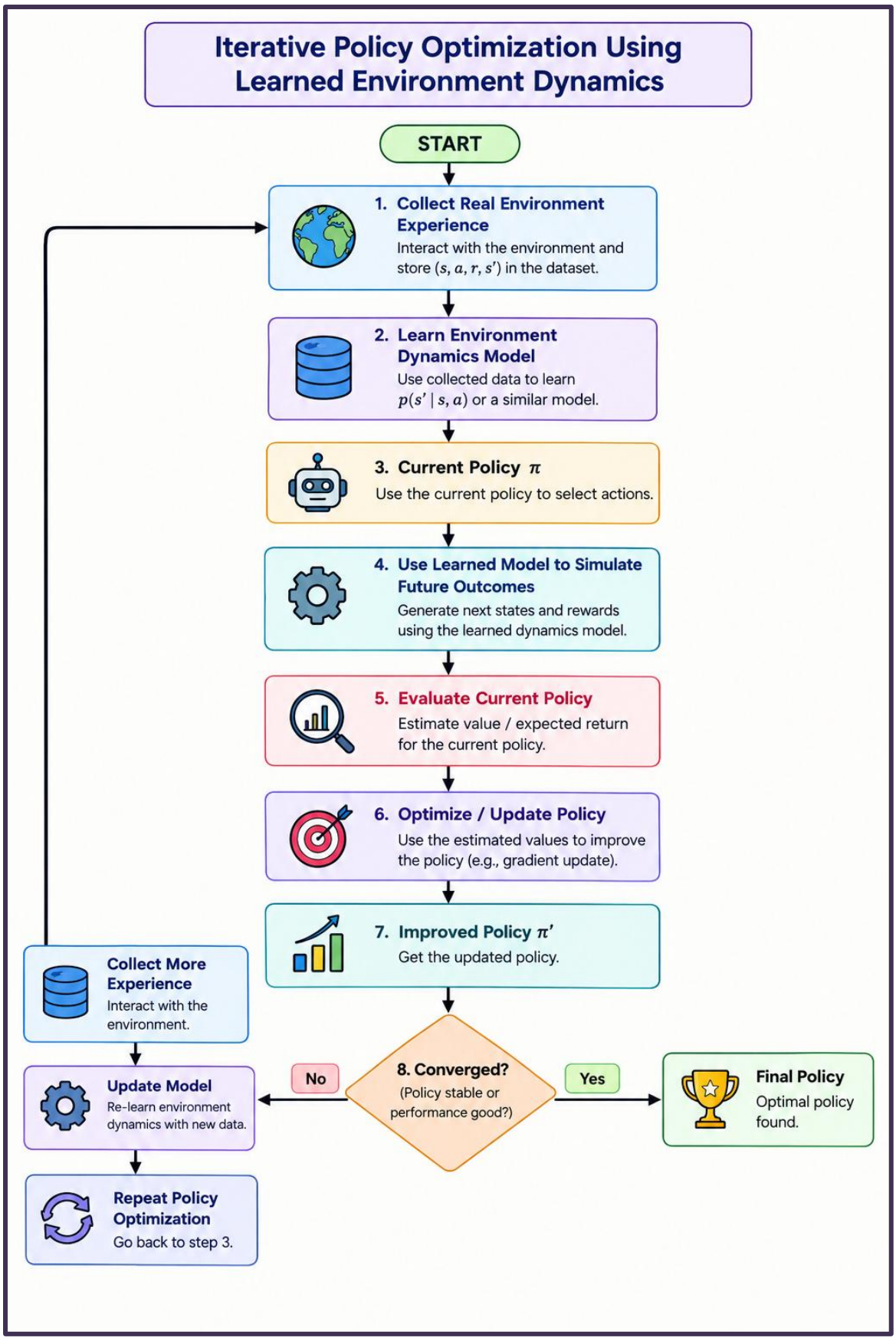
Explanation

1. **Collect Experience:** Gather data by interacting with the environment.
2. **Learn Model:** Use the data to learn environment dynamics.
3. **Simulate:** Use the learned model to predict future outcomes.
4. **Evaluate Policy:** Estimate the expected rewards of the current policy.
5. **Optimize Policy:** Update the policy to choose better actions.
6. **Check Convergence:** If the policy is not stable, collect more data and repeat.
7. **Final Policy:** Continue until an effective policy is obtained.

Key Flow:

Real Experience → Learn Model → Simulate → Evaluate → Optimize Policy → Update Model → Repeat

Flowchart

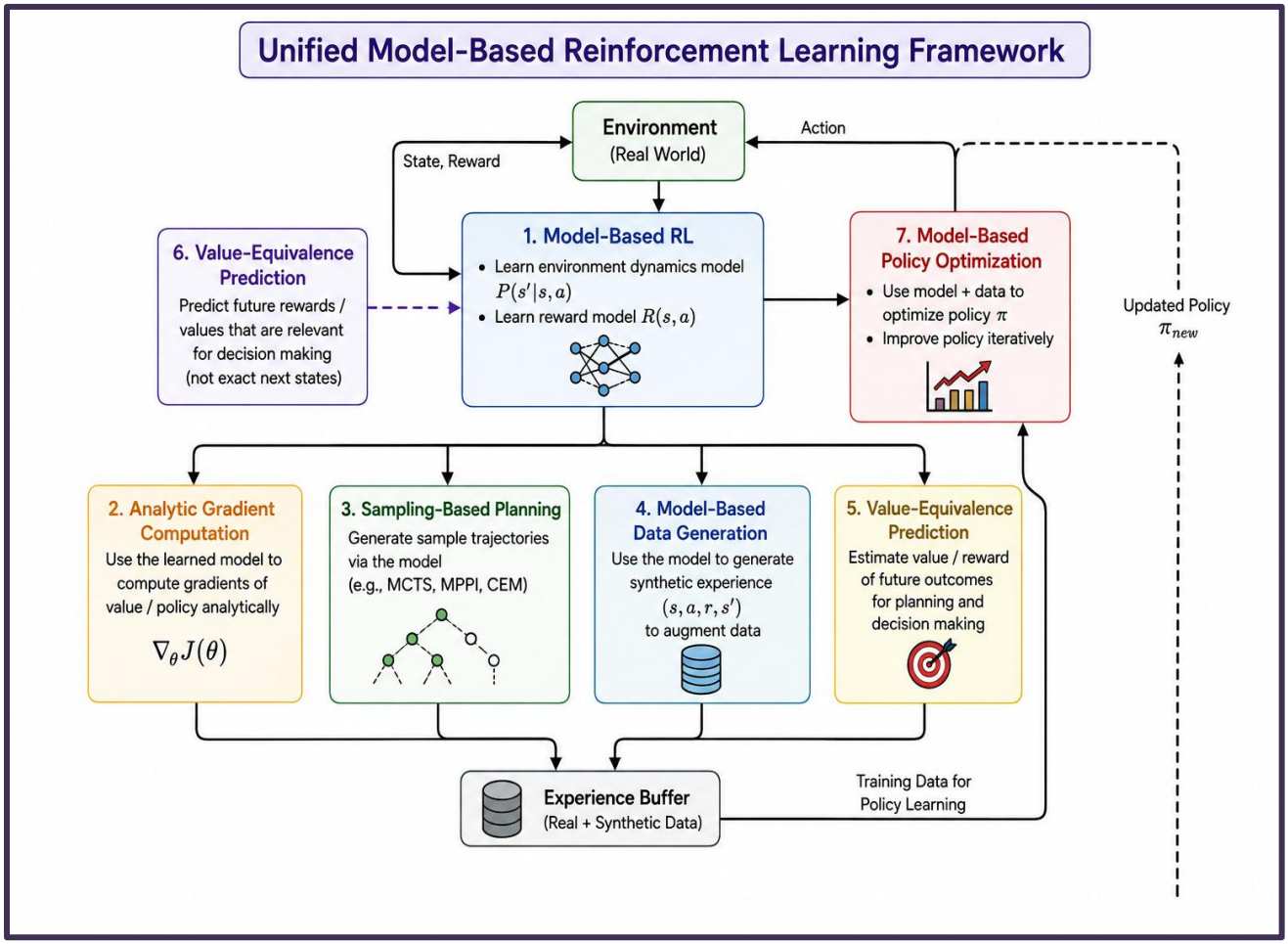

Iterative Policy Optimization Using Learned Environment Dynamics
START
1. Collect Real Environment Experience
Interact with the environment and store (s, a, r, s') in the dataset.
2. Learn Environment Dynamics Model
Use collected data to learn $p(s' \mid s, a)$ or a similar model.
3. Current Policy π
Use the current policy to select actions.
4. Use Learned Model to Simulate Future Outcomes
Generate next states and rewards using the learned dynamics model.
5. Evaluate Current Policy
Estimate value / expected return for the current policy.
6. Optimize / Update Policy
Use the estimated values to improve the policy (e.g., gradient update).
7. Improved Policy π'
Get the updated policy.
8. Converged?
(Policy stable or performance good?)
No
Yes
Final Policy
Optimal policy found.
Collect More Experience
Interact with the environment.
Update Model
Re-learn environment dynamics with new data.
Repeat Policy Optimization
Go back to step 3.

25. Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

Small Information

A **unified Model-Based RL framework** combines environment modeling, planning, data generation, value prediction, gradient computation, and policy optimization. These components work together to learn an effective policy while reducing dependence on real-environment interaction.

Concept Map



Explanation

- **Model-Based RL:** Learns a model of environment dynamics and rewards.
- **Analytic Gradient Computation:** Calculates gradients to guide policy/value updates.
- **Sampling-Based Planning:** Samples possible future trajectories and selects promising actions.

- **Model-Based Data Generation:** Uses the learned model to create synthetic experiences.
- **Value-Equivalence Prediction:** Predicts future values/rewards relevant to decision-making.
- **Model-Based Policy Optimization:** Combines model predictions and generated data to improve the policy.
- **Experience Buffer:** Stores real and synthetic experiences for training.

Overall Relationship

Environment → Model-Based RL → Planning + Gradient Computation + Data Generation + Value Prediction → Experience/Data → Policy Optimization → Improved Policy → Environment